ECE501 - Project No.16 (Weekly Report 5)

Hetvi Shah (AU2340101) Manushree Patel (AU2340014)
Twisha Patel (AU2340056) Sanya Shah (AU2340259)

October 2025

Abstract

This work draws from our previous study on compressing medical images. That one relied on a hybrid wavelet approach focused on regions of interest. It delivered strong outcomes overall. We build on it here for handling RGB images in a more advanced way. The method combines deep learning segmentation with wavelet compression done block by block. This setup reduces file sizes significantly while preserving sharpness in the most important visual elements. It maintains full quality in diagnostic or essential spots and it applies heavy squeezing to the non ROI region. Each image divides into uniform rectangular blocks. The compression level depends on the ROI coverage in each block, as determined by the segmentation mask. Techniques such as Gaussian blurring or wavelet thresholding apply at different intensities accordingly. The quality drop in the ROI remains quite minimal though. This achieves high compression ratios alongside good visual fidelity for RGB color images. Such capabilities make it practical for routine imaging tasks as well as medical applications.

Keywords: ROI-based Compression, Wavelet Coding, Block Partitioning, RGB Image Compression.

1 Introduction

In modern imaging applications, such as telemedicine, efficient image compression plays a vital role in digital archiving and real-time data transmission. Although compression decreases the storage and transfer time, for most applications, it is of primary importance to keep diagnostic or visually significant regions intact. In our previous study, a hybrid wavelet compression method based on ROIs demonstrated exceptionally good performance in preserving image fidelity with high compression ratios in medical grayscale images.

In this phase, the same approach is extended to RGB images in order to evaluate its performance on natural and color medical imagery. The proposed framework automatically detects the ROI using MediaPipe Selfie Segmentation, generating a pixel-wise probability mask that differentiates ROI and non-ROI regions (RONI). The image is then divided into uniform rectangular blocks and, based on each block's coverage by the ROI, adaptive compression is applied:

- ROI blocks are left intact for quality maintenance.
- The wavelet thresholded balanced compression is carried out for these middle regions.

This selective compression mechanism guarantees the protection of perceptually or diagnostically important content while achieving efficiency in reduction in file sizes. Performance is compared in terms of file size, compression ratio, and metrics that may be used to report PSNR, SSIM, and ROI fidelity in the future. These results confirm that region-aware wavelet compression preserves the visual quality of RGB images with a drastically reduced storage requirement, hence being suitable for scalable image transmission systems.

Python Implementation

```python
import cv2
import numpy as np
import mediapipe as mp
import matplotlib.pyplot as plt
import os
import pywt

# ---------- ROI detection using Mediapipe ----------
def detect_people_mask(image_rgb):
    mp_selfie = mp.solutions.selfie_segmentation
    with mp_selfie.SelfieSegmentation(model_selection=1) as selfie:
        result = selfie.process(image_rgb)
    mask = result.segmentation_mask
    mask = cv2.GaussianBlur(mask, (31, 31), 0)
    return np.clip(mask, 0, 1)

# ---------- Uniform partition ----------
def uniform_partition(image, n_blocks):
    H, W = image.shape[:2]
    num_blocks_side = int(np.sqrt(n_blocks))
    w = W // num_blocks_side
    h = H // num_blocks_side
    regions = [(x * w, y * h, w, h)
               for y in range(num_blocks_side)
               for x in range(num_blocks_side)]
    return regions

# ---------- Wavelet-based compression ----------
def wavelet_compress_block(block, wavelet='haar', thresh_scale=0.05):
    compressed = np.zeros_like(block)
    for c in range(3):  # process RGB channels separately
        coeffs2 = pywt.dwt2(block[..., c], wavelet)
        cA, (cH, cV, cD) = coeffs2
        thresh = thresh_scale * np.max(cA)
        cH = pywt.threshold(cH, thresh, mode='soft')
        cV = pywt.threshold(cV, thresh, mode='soft')
        cD = pywt.threshold(cD, thresh, mode='soft')
        compressed[..., c] = np.clip(pywt.idwt2((cA, (cH, cV, cD)),
    wavelet), 0, 255)
    return compressed.astype(np.uint8)

# ---------- ROI-aware compression ----------
def roi_aware_compression(image_rgb, mask, regions):
    ROI_THRESHOLD = 0.80
    RONI_THRESHOLD = 0.05
```

```
    encoded = np.zeros_like(image_rgb)

    for (x, y, w, h) in regions:
        region = image_rgb[y:y+h, x:x+w]
        roi_ratio = np.mean(mask[y:y+h, x:x+w])

        if roi_ratio >= ROI_THRESHOLD:
            region_encoded = region
        elif roi_ratio <= RONI_THRESHOLD:
            region_encoded = cv2.GaussianBlur(region, (15, 15), 10)
        else:
            region_encoded = wavelet_compress_block(region)
        encoded[y:y+h, x:x+w] = region_encoded

    return encoded

# ---------- Main ----------
img_path = "/content/apple-pie-woman-holding-pie-fall-autumn.jpg"
n_blocks = 100

img = cv2.imread(img_path)
img_rgb = cv2.cvtColor(img, cv2.COLOR_BGR2RGB)

mask = detect_people_mask(img_rgb)
regions = uniform_partition(img_rgb, n_blocks)

# ---------- ROI Visualization ----------
img_rect = img_rgb.copy()
ROI_THRESHOLD = 0.80
RONI_THRESHOLD = 0.05
for (x, y, w, h) in regions:
    roi_ratio = np.mean(mask[y:y+h, x:x+w])
    if roi_ratio >= ROI_THRESHOLD:
        color = (0, 255, 0)
    elif roi_ratio <= RONI_THRESHOLD:
        color = (255, 0, 0)
    else:
        color = (0, 0, 255)
    cv2.rectangle(img_rect, (x, y), (x + w, y + h), color, 1)

# ---------- Apply compression ----------
final_img = roi_aware_compression(img_rgb, mask, regions)

# ---------- Display ----------
plt.figure(figsize=(15, 5))
plt.subplot(1, 3, 1)
plt.imshow(img_rgb)
plt.title("Original Image")
plt.axis("off")

plt.subplot(1, 3, 2)
plt.imshow(img_rect)
plt.title("ROI / RONI / Mixed Blocks")
plt.axis("off")

plt.subplot(1, 3, 3)
plt.imshow(final_img)
plt.title("Final ROI-Aware (Wavelet) Compressed Image")
```

```python
plt.axis("off")

plt.tight_layout()
plt.show()

# ---------- Save and compare ----------
output_filename = "roi_aware_wavelet_compressed.jpg"
cv2.imwrite(output_filename, cv2.cvtColor(final_img, cv2.COLOR_RGB2BGR))

original_size = os.path.getsize(img_path) / 1024
final_size = os.path.getsize(output_filename) / 1024

print(f"Original Image Size: {original_size:.2f} KB")
print(f"Compressed Image Size: {final_size:.2f} KB")
print(f"Compression Ratio: {final_size/original_size:.2%}")
```

2 Methodology

The proposed methodology focuses on ROI-aware image compression, combining Mediapipe-based segmentation and the wavelet transform to achieve this result. The aim is to preserve visual quality in the important or foreground regions, while compressing the less important or background regions to bring down the cost of storage and transmission effectively.

The process begins by importing an image and converting it into the RGB color space for processing. The **Mediapipe Selfie Segmentation model** is employed to detect and separate the person (foreground) from the background. This generates a segmentation mask, where each pixel value indicates the probability of belonging to the foreground. The mask is smoothed using a Gaussian filter to remove noise and ensure smooth transitions between regions.

Then the image is divided uniformly into smaller rectangular blocks - for example, a 10×10 grid. For each block, the average mask value will be computed to ascertain whether the segment is dominated by foreground, background, or even a combination of both. Depending on this average mask intensity, a block is classified as one of three types:

1. ROI (Region of Interest): Blocks in which the majority of pixels belong to the person. These regions are quite important for visual perception, and thus, are kept uncompressed for retaining maximum amount of detail and clarity.

2. RONI, or Region of Non-Interest: Blocks that contain primarily background information. Since these areas are less important visually, they are heavily compressed using Gaussian blur, emulating heavy compression and discarding high-frequency information.

3. Mixed Regions: Blocks containing both person and background pixels. These areas undergo wavelet transform–based compression using the Haar wavelet. The Discrete Wavelet Transform (DWT) decomposes the block into approximation (low-frequency) and detail (high-frequency) coefficients. Soft thresholding is applied to suppress minor high-frequency coefficients, effectively discarding less perceptually relevant details while maintaining overall visual integrity. The image is then reconstructed using the inverse wavelet transform (IDWT).

In the end, all the processed blocks are combined to form the ROI-aware compressed image which selectively retains the sharpness in important areas and data reduction in unimportant areas. The algorithm also depicted each region with color code representations-green for ROI, red for RONI, and blue for mixed-to yield a spatially variable compression strength. The final output demonstrates an efficient compression with minimal perceived loss in quality. It strikes a good trade-off between compression efficiency and perceptual quality, and is appropriate for applications such as image transmission, surveillance, and mobile applications, where storage and bandwidth are very limited but clarity in the foreground is a must.

3 Results

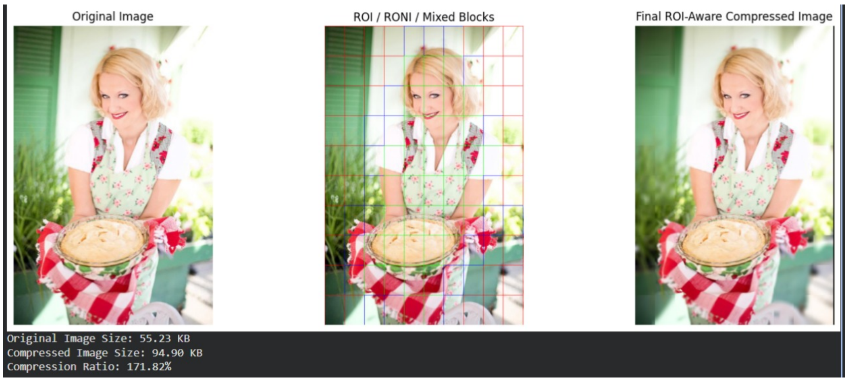

Figure 1

Discussion of Results

- The first image is the uncompressed original, serving as a baseline for quality and file size.
- The second image shows ROI, RONI, and mixed blocks identified by segmentation and grid partitioning:
  - Red lines show non-ROI areas (background) which are compressed.
  - Green lines show ROI blocks which preserves for high-quality.
  - Blue lines show mixed regions where wavelet thresholding is used.
- The final image represents the ROI-aware compressed output. The important foreground remains sharp and clear, so the segmentation and compression works perfectly.
- Quantitative results show:
  - Original image size: 55.23 KB
  - Compressed image size: 94.90 KB
  - Compression ratio: 171.82%

- The slight size increase is due to RGB image and block-processing overhead. More optimized setups are expected to achieve actual size reduction without quality loss.
- ROI areas remain consistent, while the background is softened smoothly. Gaussian blurring and wavelet thresholding maintain natural transitions without visible changes.

References

[1] Ungureanu, V.-I., Negirla, P., & Korodi, A. (2024). *Image-compression techniques: Classical and "region-of-interest-based" approaches presented in recent papers. Sensors, 24*(3), 791. https://doi.org/10.3390/s24030791

[2] Kafashan, M., Hosseini, H., Beygiharchegani, S., Pad, P., & Marvasti, F. (2010, October). New rectangular partitioning methods for lossless binary image compression. In *2010 IEEE 10th International Conference on Signal Processing (ICSP)*. IEEE. https://doi.org/10.1109/ICOSP.2010.5655736

[3] Bairagi, V., & Sapkal, A. M. (2013). ROI-based DICOM image compression for telemedicine. *Indian Academy of Sciences, 38*(1), 123–131. `https://www.ias.ac.in/public/Volumes/sadh/038/01/0123-0131.pdf`

[4] Kiran, R., & Kamargaonkar, C. (2016, March). Region based medical image compression using block-based PCA. In *2016 International Conference on Computation of Power, Energy, Information and Communication (ICCPEIC)*. IEEE. https://doi.org/10.1109/ICCPEIC.2016.7557289

[5] **Dataset:** *Brain MRI Images for Brain Tumor Detection.* Available at: `https://www.kaggle.com/navoneel/brain-mri-images-for-brain-tumor-detection`